

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG



BÁO CÁO THỰC HÀNH

LAB 01: TÍNH TOÁN BIT

Manipulating Bits – Tính toán trên bit

Nhóm: Nhóm 4

Sinh viên 1: Bùi Thái Sơn – 25521588

Sinh viên 2: Võ Quốc Thịnh – 25521787

Sinh viên 3: Lê Thành Đạt – 25520270

Thành phố Hồ Chí Minh, tháng 9 năm 2026

Mục lục

1	Tính toán bit đơn giản	2
1.1	Hàm negative	2
1.2	Hàm cal100x	2
1.3	Hàm flipByte	2
1.4	Hàm getnbit	2
1.5	Hàm round2n	3
2	Các phép kiểm tra dựa trên tính toán bit	3
2.1	Hàm isSameSign	3
2.2	Hàm isPositive	3
2.3	Hàm isMulpw2	4
2.4	Hàm isLess2n	4
2.5	Hàm logicNot	4
3	Minh chứng kết quả thực thi	5

1 Tính toán bit đơn giản

1.1 Hàm negative

Yêu cầu: Tính $-x$ mà không dùng toán tử trừ.

Trong biểu diễn bù 2, số đối của x được tạo bằng cách đảo tất cả các bit rồi cộng 1:

$$-x = \sim x + 1.$$

```
1 int negative(int x)
2 {
3     return ~x + 1;
4 }
```

1.2 Hàm cal100x

Yêu cầu: Tính $100x$ mà không dùng toán tử nhân.

Ta phân tích $100 = 64 + 32 + 4 = 2^6 + 2^5 + 2^2$. Do đó phép nhân với 100 được thay bằng ba phép dịch trái và hai phép cộng:

$$100x = (x \ll 6) + (x \ll 5) + (x \ll 2).$$

```
1 int cal100x(int x)
2 {
3     return (x << 6) + (x << 5) + (x << 2);
4 }
```

1.3 Hàm flipByte

Yêu cầu: Lật 1 Byte thứ n của số nguyên x (các byte được đánh thứ tự từ 0 đến 3 từ phải sang trái), từ bit 0 thành 1 và ngược lại.

Mỗi byte gồm 8 bit. Byte thứ n bắt đầu tại vị trí $8n$, vì vậy mask cần dùng là $0xff \ll (n \ll 3)$. Phép XOR với bit 1 làm đảo bit tương ứng, còn XOR với bit 0 giữ nguyên bit ban đầu.

```
1 int flipByte(int x, int n)
2 {
3     int mask = 0xff << (n << 3);
4     return x ^ mask;
5 }
```

1.4 Hàm getnbit

Yêu cầu: Trả về n bit thấp nhất (bên phải) của số nguyên x (các bit được đánh số thứ tự từ 0 đến 31 từ phải sang trái). Mức nâng cao nhận mọi $n \geq 0$ và trả về tối đa 32 bit.

Mask gồm n bit 1 ở bên phải được tạo bằng cách dịch một dãy bit 1 sang trái n vị trí rồi đảo lại. Thực hiện AND với x sẽ loại bỏ tất cả các bit từ vị trí n trở lên. Hằng 0LL mở rộng phép tạo mask lên 64 bit, nhờ đó các ca kiểm thử nâng cao với $n \geq 32$ vẫn giữ được toàn bộ 32 bit của x .

```

1 unsigned getnbit(unsigned x, int n)
2 {
3     int mask_lastn = (~(0LL) << n);
4     return x & mask_lastn;
5 }

```

1.5 Hàm round2n

Yêu cầu: Làm tròn xuống x đến bội gần nhất của 2^n . Mức nâng cao yêu cầu làm tròn lên hoặc xuống dựa trên việc phần dư đã đạt một nửa khoảng 2^n hay chưa.

Để làm tròn x đến bội gần nhất của 2^n , trước hết cộng thêm một nửa khoảng làm tròn là 2^{n-1} , sau đó xóa n bit thấp bằng một cặp dịch phải–dịch trái:

$$\text{round}_{2^n}(x) = ((x + 2^{n-1}) \gg n) \ll n.$$

```

1 int round2n(int x, int n)
2 {
3     int half = (1 << n) >> 1;
4     return ((x + half) >> n) << n;
5 }

```

2 Các phép kiểm tra dựa trên tính toán bit

2.1 Hàm isSameSign

Yêu cầu: Trả về 1 nếu x và y cùng âm hoặc cùng không âm; ngược lại trả về 0.

Bit dấu nằm ở vị trí 31. Sau khi dịch phải 31 bit và AND với 1, ta thu được bit dấu của mỗi số. Hai số cùng dấu khi XOR hai bit dấu bằng 0; phủ định logic kết quả XOR sẽ trả về đúng 1 trong trường hợp này.

```

1 int isSameSign(int x, int y)
2 {
3     int sign_x = (x >> 31) & 0x1;
4     int sign_y = (y >> 31) & 0x1;
5     return !(sign_x ^ sign_y);
6 }

```

2.2 Hàm isPositive

Yêu cầu: Trả về 1 khi $x > 0$, bao gồm việc trả về 0 cho cả số âm và số 0.

Một số dương phải đồng thời khác 0 và có bit dấu bằng 0. Biểu thức bên trong trả về khác 0 nếu $x = 0$ hoặc $x < 0$; phép ! ngoài cùng chuẩn hóa kết quả về 0 hoặc 1.

```

1 int isPositive(int x)
2 {
3     return !(!x | ((x >> 31) & 0x1));
4 }

```

2.3 Hàm isMulpw2

Yêu cầu: Kiểm tra x có chia hết cho 2^n hay không, với x tùy ý và n không âm.

x chia hết cho 2^n khi n bit thấp nhất đều bằng 0. Cặp dịch $(x \gg n) \ll n$ xóa n bit thấp; nếu giá trị sau khi xóa vẫn bằng x thì XOR của hai giá trị bằng 0.

```
1 int isMulpw2(int x, int n)
2 {
3     return !((x >> n) << n ^ x);
4 }
```

2.4 Hàm isLess2n

Yêu cầu: Trả về 1 nếu $x < 2^n$. Input: x nguyên dương và $0 \leq n \leq 30$.

Với x không âm, điều kiện $x < 2^n$ tương đương với việc mọi bit từ vị trí n trở lên đều bằng 0. Vì vậy $x \gg n$ phải bằng 0.

```
1 int isLess2n(int x, int n)
2 {
3     return !((x >> n) << n);
4 }
```

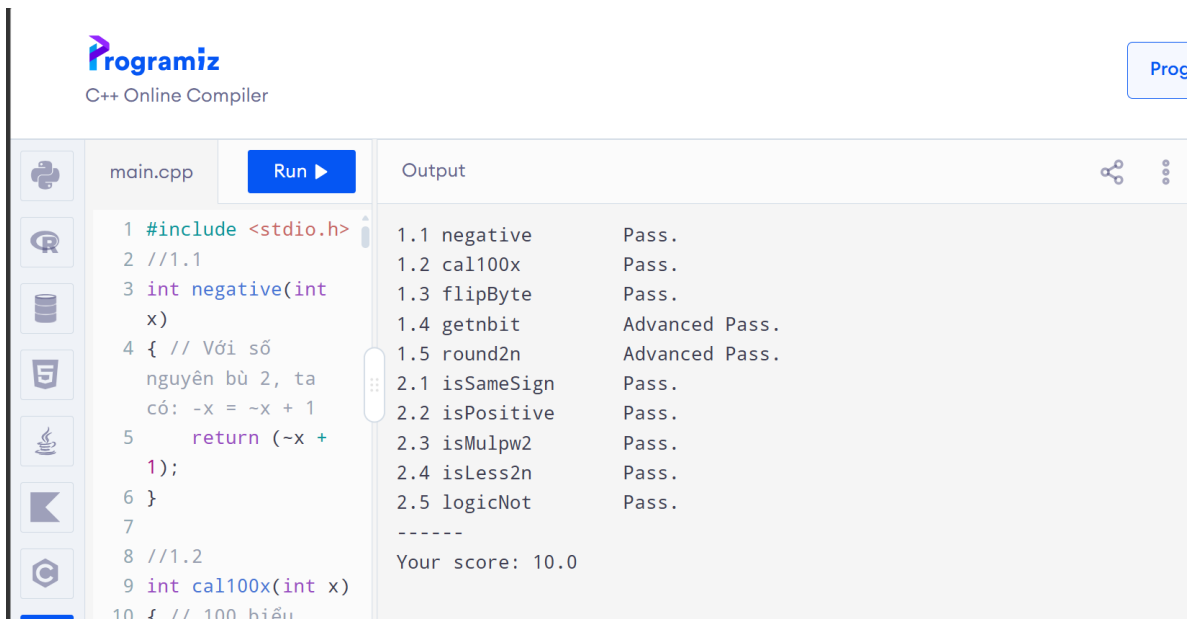
2.5 Hàm logicNot

Yêu cầu: Cài đặt kết quả của $!x$ mà không sử dụng toán tử $!$, áp dụng cho toàn bộ miền giá trị của kiểu `int`.

Nếu $x \neq 0$, ít nhất một trong hai giá trị x và $-x$ có bit dấu bằng 1. Bởi vậy $x \mid -x$ có bit dấu bằng 1; dịch phải số học 31 bit cho kết quả $0xffffffff$, tức -1 . Riêng với $x = 0$, kết quả sau khi dịch là 0. Lấy giá trị 2 trường hợp này $+1$ ta được giá trị cần trả về.

```
1 int logicNot(int x)
2 {
3     int neg_x = ~x + 1;
4     int has_sign = (x | neg_x) >> 31;
5     return has_sign + 1;
6 }
```

3 Minh chứng kết quả thực thi



The screenshot displays the Programiz C++ Online Compiler interface. The code in `main.cpp` includes `<stdio.h>` and defines two functions: `negative` and `cal100x`. The `negative` function takes an integer `x` and returns `~x + 1`. The `cal100x` function takes an integer `x` and returns `x`. The program tests various bit manipulation functions: `negative`, `cal100x`, `flipByte`, `getnbit`, `round2n`, `isSameSign`, `isPositive`, `isMulpw2`, `isLess2n`, and `logicNot`. The output shows that all tests passed, resulting in a score of 10.0.

```
1 #include <stdio.h>
2 //1.1
3 int negative(int x)
4 { // Với số
  nguyên bù 2, ta
  có: -x = ~x + 1
5   return (~x +
6   1);
7 }
8 //1.2
9 int cal100x(int x)
10 { // 100 biểu
```

Output:

```
1.1 negative      Pass.
1.2 cal100x       Pass.
1.3 flipByte      Pass.
1.4 getnbit       Advanced Pass.
1.5 round2n       Advanced Pass.
2.1 isSameSign    Pass.
2.2 isPositive    Pass.
2.3 isMulpw2      Pass.
2.4 isLess2n      Pass.
2.5 logicNot      Pass.
-----
Your score: 10.0
```

Hình 1: Kết quả biên dịch và chạy chương trình trên Programiz.